競技プログラミング アルゴリズム早見表（制約 × アルゴリズム × 解法）

ICPC 国内予選（E 問題ライン）～ AtCoder 水～青レベルまでを想定した汎用リファレンス。

1. 制約から計算量を逆引きする表

1 秒 ≈ 10^8 回の単純演算が基準。ICPC 国内予選は手元実行のため実行時間制限が事実上なく、**1〜2 桁重くても「回せば通る」**。

N の上限	許容計算量	主な選択肢
N ≤ 8	O(N! × N^2) 以上も可	順列全探索＋重い検証、集合の全分割の列挙（ベル数 B8=4140）
N ≤ 11	O(N!)	順列全探索（next_permutation）
N ≤ 16〜18	O(2^N × N^2)	bitDP（TSP・集合分割）、部分集合全列挙、O(3^N) の部分集合の部分集合
N ≤ 20〜25	O(2^N)	bit 全探索、部分和
N ≤ 40	O(2^(N/2) × N)	半分全列挙（meet in the middle）
N ≤ 50	O(N^4)〜O(N^5)	多重ループ、フロー系
N ≤ 100	O(N^3)〜O(N^4)	Warshall–Floyd、区間DP、行列累乗、二部マッチング
N ≤ 500	O(N^3)	Warshall–Floyd、DP[N][N]×遷移N、ハンガリアン法
N ≤ 3,000	O(N^2)	全ペア列挙、LCS・編集距離、素朴 DP
N ≤ 10^4	O(N^2) ≈ 10^8（ギリギリ）	2 重ループ（国内予選なら余裕）
N ≤ 2×10^5	O(N log N)	ソート、priority_queue、二分探索、セグ木/BIT、Dijkstra、座標圧縮
N ≤ 10^7	O(N)	累積和、いもす法、尺取り法、エラトステネスの篩、線形 DP
N ≤ 10^9	O(√N)、O(log N)	試し割り、二分探索、桁ごとの処理（国内予選なら O(N) 単純ループも可）
N ≤ 10^12〜10^15	O(√N) ≈ 3×10^7	素因数分解、約数列挙、平方分割的な数え上げ
N ≤ 10^18	O(log N)、O(1)	繰り返し二乗法、行列累乗、ダブリング、桁 DP、答えの二分探索、数式変形。**long long 必須・オーバーフロー警戒**

制約文の「シグナル」の読み方

制約・文言	疑うべき手法
N ≤ 16 前後	bitDP・部分集合列挙
N ≤ 40 で部分和っぽい	半分全列挙
重みが 0/1 だけ	01-BFS
辺コストがすべて 1	BFS
負の辺・「呪い」「儲け」	Bellman–Ford（負閉路検出）
V ≤ 300〜500 で全点対	Warshall–Floyd
「最大値の最小化」「〜できる最小の x」	答えで二分探索（判定問題化）
「区間への一様な加算が大量」	いもす法・遅延セグ木
「連続部分列で条件を満たす最長/個数」	尺取り法・累積和＋二分探索
値域が 10^9 だが個数が 10^5	座標圧縮
「クエリで結合・同じグループか」	Union-Find
「K 番目」「中央値を保ちたい」	priority_queue 2 本、平衡木/multiset
mod 998244353 / 10^9+7	数え上げ DP・組合せ・逆元
操作しても変わらない量がある	不変量（GCD・偶奇・総和 mod k）
BNF・文法規則が書いてある	再帰下降構文解析
座標が小さい整数（<1000 等）	幾何の全探索、整数演算で誤差回避

2. 探索

アルゴリズム	使いどころ（目印）	計算量	解法の要点
全探索（多重ループ）	候補の直積が 10^8 以下	積	「答えを決め打ちして検証できるか」から考える
順列全探索	並べ方・割り当て、N ≤ 11	O(N!·検証)	C++ は `next_permutation`（ソート済みから開始）
bit 全探索	選ぶ/選ばない、N ≤ 25	O(2^N·検証)	`for(int S=0;S<(1<<n);S++)`、`S>>i&1`
部分集合の部分集合	集合を 2 グループに割る、N ≤ 16	O(3^N)	`for(int T=S;;T=(T-1)&S){...if(!T)break;}`
半分全列挙	部分和・組合せで N ≤ 40	O(2^(N/2)·N)	前半後半に分けて列挙→片方をソートし二分探索/尺取り

アルゴリズム	使いどころ（目印）	計算量	解法の要点
DFS（バックトラック）	全構成の列挙、制約充足	指数（枝刈り次第）	「これ以上良くならない」上界/下界での枝刈りが本体
BFS	最小手数・最短ステップ	O(状態数+遷移)	状態をエンコード（tuple→map、盤面→文字列/整数）
反復深化・双方向探索	状態数が莫大で深さが浅い	O(b^(d/2))	双方向 BFS は往復から中間で衝突させる
答えで二分探索	単調性のある最適化	O(log(範囲)·判定)	「x で可能なら x±1 でも可能」を確認してから書く。実数は 100 回固定ループ
三分探索	凸（単峰）な関数の最小/最大	O(log·評価)	整数なら区間幅 3 未満まで、実数なら 200 回
周期・サイクル検出	決定的な状態遷移の繰り返し	O(周期長)	set/map に状態を記録、再訪で周期確定。巨大回数 K は「前処理＋周期で割る」

3. 動的計画法（DP）

型	目印	状態・遷移	計算量
線形 DP	「i 番目まで見て最適」	dp[i] または dp[i][直前の状態]	O(N)〜O(NK)
ナップサック	重さ・容量・予算	dp[i][w] = 価値最大。個数無制限は w 昇順で in-place	O(NW)
部分和・個数	「ちょうど S にできるか/何通り」	dp[i][s] = bool/通り数	O(NS)
複数リソース分割	「2〜3 グループに分ける」	dp[i][x][y]（残り 1 つは総和から復元）	O(N·S^2)
LCS / 編集距離	文字列 2 本の類似度	dp[i][j]、遷移 3 方向	O(NM)
LIS	最長増加部分列	dp[長さ]=末尾最小値、lower_bound で更新	O(N log N)
区間 DP	「両端から消す」「連結・合成」	dp[l][r] = 区間 [l,r) の最適。幅の短い順	O(N^3)
bitDP	訪問済み集合、順序付き割り当て N≤17	dp[S][v] = 集合 S を使い終え最後が v	O(2^N N^2)
桁 DP	「N 以下で条件を満たす個数」N≤10^18	dp[桁][N未満確定?][条件の状態]、上の桁から	O(桁×状態×10)
木 DP	木の各部分木で最適	dfs で子の値を合成。「根を変える」は全方位木 DP	O(N)〜O(N^2)

型	目印	状態・遷移	計算量
期待値 DP	「期待値を求めよ」	後ろから dp[状態]=Σp·(dp[次]+コスト)。自己ループは式変形で解く	状態×遷移
確率 DP	「確率を求めよ」	前から確率を配る	状態×遷移
ゲーム DP（後退解析）	二人交互・最適に行動	win/lose をメモ化再帰。「負けに遷移できれば勝ち」	状態×遷移
数え上げ DP + mod	「場合の数を 10^9+7 で」	加算のたび mod。引き算は +MOD。除算は逆元	—
戻す DP / 挿入 DP / 箱根駅伝型	上級（E 超級）	必要になったら学ぶ	—

設計手順: ①状態 =「未来の判断に必要な情報の最小集合」 ②遷移 ③初期値 ④答えの位置。迷ったらメモ化再帰で書いてから配列 DP に直す。

---

4. グラフ

アルゴリズム	目印	計算量	解法の要点
DFS / BFS	連結性、到達可能性、二部判定	O(V+E)	二部判定は 2 色塗り分けの矛盾検出
01-BFS	辺コストが 0/1	O(V+E)	deque。コスト 0 は前に、1 は後ろに push
Dijkstra	非負コスト最短路	O(E log V)	priority_queue（距離, 頂点）。取り出し時に古い要素を捨てる
状態拡張 Dijkstra/BFS	燃料・切符・乗換回数など追加条件	状態数×log	頂点を (v, 残量) に拡張。指数的に伸びるコストは「上限回数」で打ち切れないか考える
Bellman–Ford	負辺あり、負閉路検出	O(VE)	V 回目に更新があれば負閉路。到達可能性と組み合わせる
Warshall–Floyd	全点対、V ≤ 500	O(V^3)	`d[i][j]=min(d[i][j],d[i][k]+d[k][j])`、k が最外ループ。経由可否・推移閉包にも
Union-Find	動的な連結クエリ、クラスカル	ほぼ O(α)	経路圧縮＋サイズ併合。**二部/敵味方判定は頂点 2 倍化（v と v+N）**
最小全域木（Kruskal / Prim）	「全部つなぐ最小コスト」	O(E log E)	Kruskal = 辺ソート＋UF。「最大コスト辺の最小化」も MST
トポロジカルソート	DAG、依存関係、閉路検出	O(V+E)	入次数 0 をキューに。全部処理できなければ閉路あり
強連結成分分解（SCC）	有向グラフの相互到達	O(V+E)	縮約後は DAG → DP と組み合わせ。2-SAT の基盤

アルゴリズム	目印	計算量	解法の要点
橋・関節点（lowlink）	「切れると分断される辺/点」	O(V+E)	DFS 木の ord/low
LCA	木上の 2 点間クエリ	O(N log N) 前処理	ダブリング。dist(u,v)=d[u]+d[v]−2d[lca]
木の直径	木の最遠点対	O(N)	任意点から最遠の点 u → u から最遠の点 v
最大流（Dinic）	割り当て・分断の最小コスト、V ≤ 500	O(V^2 E)（実用は速い）	最小カット＝最大流。「選べない組合せ」は燃やす埋める
二部マッチング	仕事の割り当て、V ≤ 1000	O(VE)	増加路を DFS で。最大マッチング＝頂点数−最大独立集合（König）
最小費用流	コスト付き割り当て	O(FE log V)	SPFA/Dijkstra(ポテンシャル)で最短路を流す
オイラー路	「全辺を一度ずつ」	O(E)	奇数次数の頂点が 0 個（回路）or 2 個（路）

5. データ構造

構造	目印	計算量	要点
stack / queue / deque	括弧対応、直近の要素、スライド最小値	O(1)/操作	スライド最小値は単調 deque
priority_queue	「毎回最小(最大)を取り出す」	O(log N)	Dijkstra、貪欲（会議室・マージコスト）
set / multiset / map	順序付き集合、「x 以上の最小」	O(log N)	`lower_bound`。multiset の erase は `s.erase(s.find(x))`
BIT（Fenwick）	一点加算＋区間和、転倒数	O(log N)	転倒数 = 右から「自分より小さい既出数」を数える
セグメント木	区間 min/max/和 ＋ 一点更新	O(log N)	演算はモノイドなら何でも。遅延評価版で区間更新
累積和 / 2 次元累積和	区間和クエリが大量（更新なし）	O(1)/クエリ	imos 法：区間加算を差分で記録して最後に累積
尺取り法	連続部分列＋単調な条件	O(N)	右を伸ばし、条件が壊れたら左を縮める
座標圧縮	値域 10^9・個数 10^5	O(N log N)	sort → unique → lower_bound で番号化
ダブリング	「K 回遷移した先」K ≤ 10^18	O(N log K)	next[k][v] = 2^k 回後。LCA・周期処理に

構造	目印	計算量	要点
ローリングハッシュ	部分文字列の一致判定を大量に	O(1)/比較	衝突対策に mod/基数を 2 組
Trie	大量の文字列のプレフィックス	O(長さ)	XOR 最大化は 2 進 Trie

---

6. 文字列

アルゴリズム	目印	計算量	要点
素朴一致・std::string::find	長さが小さい	O(NM)	国内予選レベルはこれで足りることが多い
KMP / Z-algorithm	パターン検索・境界（周期）	O(N+M)	Z[i] = S と S[i..] の最長共通接頭辞
ローリングハッシュ	一致・回文・辞書順比較の高速化	O(N) 前処理	二分探索＋ハッシュで最長共通接頭辞
Manacher	最長回文	O(N)	中心ごとの回文半径
Suffix Array	部分文字列の辞書順・出現回数（上級）	O(N log N)	E 超級。必要時のみ
構文解析（再帰下降）	BNF・数式・入れ子構造	O(N)	下記テンプレ参照

再帰下降構文解析テンプレ（国内予選頻出）

```
// <expr> ::= <term> { ('+'|'-') <term> }
// <term> ::= <factor> { ('*'|'/') <factor> }
// <factor> ::= 数 | '(' <expr> ')'
int pos; string s;
long long expr(){ long long v=term();
  while(pos<s.size()&&(s[pos]=='+'||s[pos]=='-')){char op=s[pos++]; v = op=='+'?
v+term(): v-term();}
  return v; }
```

- 非終端記号 1 つ = 関数 1 つ。pos を進めながら再帰。
- 値そのものでなく「(min, max)」「{小,等,大} の 3 値」「集合」など**抽象化した値**で評価する変種が頻出。
- 木を明示的に作ってから木 DP する方が見通しが良い場合もある。

---

7. 数学・数論

アルゴリズム	目印	計算量	要点
GCD / LCM	「等分」「周期の一致」「操作の不変量」	O(log)	`lcm(a,b)=a/gcd(a,b)*b`（オーバーフロー順序注意）
拡張ユークリッド	ax+by=gcd の整数解、mod 逆元	O(log)	解の存在 ⇔ gcd(a,b)
エラトステネスの篩	N ≤ 10^7 の素数列挙・最小素因数	O(N log log N)	最小素因数表を持つと素因数分解が O(log)
試し割り素因数分解	N ≤ 10^15 程度の 1 個の数	O(√N)	√N まで割り、残りが 1 でなければそれも素数
約数列挙	「積に分解」「w×d×h」	O(√N)	i ≤ √N で i と N/i をペアで
繰り返し二乗法	a^b mod p、b ≤ 10^18	O(log b)	行列に載せれば線形漸化式の第 N 項（フィボナッチ等）
mod 逆元・nCr	数え上げ mod 素数	前処理 O(N)	階乗と逆階乗のテーブル。**mod なしの nCr はパスカルの三角形＋ long long / 多倍長（Python）**
包除原理	「少なくとも 1 つ〜」の補集合、条件 ≤ 20	O(2^n)	符号 (−1)^
鳩の巣原理	「必ず存在する」証明系	—	累積和 mod N が一致する 2 点、など
確率・期待値	線形性	—	E[X+Y]=E[X]+E[Y] は独立でなくても成立
不変量	「操作を繰り返すと最後どうなる」	—	GCD・総和の偶奇・mod k・転倒数の偶奇を疑う

8. 幾何（国内予選で特に頻出）

部品	用途	要点
外積 cross(a,b)=ax·by−ay·bx	向き判定（CCW）、面積	符号で左折/右折/一直線。**整数座標なら整数のまま比較して誤差回避**
内積 dot	角度（鋭角/鈍角）、射影	点と線分の距離の場合分けに
線分交差判定	「ぶつかるか」	互いの端点が相手の両側にある（CCW×4）＋端点上の特判
点と直線/線分の距離	最接近	射影が線分内なら垂線、外なら端点距離

部品	用途	要点
多角形の面積	靴紐公式	
点の内外判定	多角形内か	半直線との交差回数の偶奇（辺上は特判）
凸包	「囲む」「一番外側」	Andrew's monotone chain O(N log N)。回転キャリパーで最遠点対
円と円/直線の交点	電波・照明・接触	距離と半径の関係で場合分け（離れる/外接/交わる/内接/含む）
回転・対称移動	折り紙、重ね合わせ	回転行列 (x cosθ − y sinθ, x sinθ + y cosθ)。直線対称は射影から折り返し
誤差設計	実数比較全般	EPS は座標スケールに合わせ 1e-6〜1e-9。「微妙なケースは入力にない」保証の有無を問題文で確認

9. 貪欲法・その他の考え方

パターン	目印	要点
区間スケジューリング	「最大何個の予定」	終了時刻の早い順に採用
交換argument	「並べる順序の最適化」	隣接 2 要素の交換で損しない順（比較関数化）にソート
マージコスト最小	「結合コスト＝和」	小さい 2 つから priority_queue で（ハフマン）
後ろから考える	最後の操作が限定される	逆操作のシミュレーション
偶奇・塗り分け	市松模様、パリティ	到達可能性・不可能性の証明に
小さいケースの実験	規則が見えない数え上げ	素朴解を書いて N=1..10 を出力 → 規則・周期・OEIS 的推測
埋め込み	入力の種類が少ない/前計算可能	ICPC は出力提出なので、重い前計算を回して表を埋め込むのも正攻法

10. 実戦チェックリスト（バグ・落とし穴）

- **複数データセット**: `while (cin >> n && n)`。グローバル配列・フラグの**初期化忘れ**が最頻出バグ。
- **オーバーフロー**: 10^5 × 10^5 = 10^10 は int 不可。掛け算する前に long long へ。
- **配列サイズ**: 制約の最大値＋余裕。1-indexed / 0-indexed の混在に注意。
- **実数出力**: 指定桁数 `printf("%.10f")`。−0.0 の出力に注意。
- **幾何**: EPS の一貫性。sqrt を避けて 2 乗で比較できないか。
- **二分探索**: 境界（ok/ng の初期値）を「絶対に成り立つ/成り立たない」値に。
- **DP**: 初期値（不可能は −INF/INF）、遷移の順序（in-place 更新の向き）。
- **グラフ**: 多重辺・自己ループ・非連結の考慮。有向/無向の読み違い。
- **提出前**: サンプル全ケース一致、最大ケースを自作して実行時間と桁あふれを確認。